

Asignación II informe de investigación y defensa

Daniel Garrido

UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

Lenguaje y compiladores 2026-I

Félix Márquez

3 de junio de 2026

Resumen Académico

El presente informe tiene como objetivo analizar distintos aspectos relacionados con los lenguajes de programación, abordando sus paradigmas, estructuras léxicas y sintácticas, rendimiento computacional y aplicaciones especializadas. En primer lugar, se desarrolló una matriz descriptiva de paradigmas de programación, identificando las características fundamentales de los enfoques imperativo, orientado a objetos, funcional, lógico y concurrente, así como la tendencia actual hacia los lenguajes multiparadigma.

Posteriormente, se realizó un análisis comparativo de lenguajes modernos mediante la ejecución de pruebas de benchmarking utilizando un algoritmo basado en la Conjetura de Collatz. Los resultados obtenidos permitieron observar diferencias significativas de rendimiento entre las implementaciones evaluadas, evidenciando la influencia de los mecanismos de compilación e interpretación sobre la eficiencia de ejecución.

Finalmente, se diseñó un Lenguaje de Dominio Específico denominado Lenguaje L, orientado a la supervisión y control de sistemas de microrredes eléctricas inteligentes dentro del entorno ECO-GRID. Para ello se definieron sus reglas léxicas, palabras reservadas, estructura sintáctica y programas operativos capaces de gestionar escenarios críticos relacionados con la seguridad térmica de baterías y la optimización energética de la red.

Introducción

Los lenguajes de programación constituyen la base del desarrollo de software moderno y permiten expresar soluciones a problemas computacionales mediante diferentes enfoques de diseño. A lo largo de su evolución han surgido diversos paradigmas que ofrecen distintas formas de estructurar programas y gestionar datos.

El estudio de los lenguajes de programación comprende aspectos léxicos y sintácticos que permiten interpretar correctamente las instrucciones escritas por los desarrolladores. Estos elementos son fundamentales para el funcionamiento de compiladores e intérpretes, así como para garantizar la correcta ejecución de los programas.

Además de su estructura interna, los lenguajes pueden evaluarse desde la perspectiva de su rendimiento y eficiencia mediante pruebas comparativas. Asimismo, es posible diseñar lenguajes especializados para dominios concretos, permitiendo una interacción más eficiente con sistemas complejos.

En este informe se analizan distintos paradigmas de programación, se estudian las características léxicas y sintácticas de varios lenguajes modernos, se realiza una comparación de rendimiento mediante benchmarking y se propone el diseño de un lenguaje de dominio específico para la gestión de microredes eléctricas inteligentes.

Actividad I. Matriz Descriptiva y Análisis de Paradigmas de Programación

Marco Analítico

Los paradigmas de programación representan distintos enfoques para el diseño y desarrollo de software. Cada paradigma define una manera particular de estructurar programas, gestionar datos y resolver problemas computacionales. A medida que la industria del software ha evolucionado, los lenguajes modernos han incorporado características de múltiples paradigmas para responder a necesidades cada vez más complejas, dando origen a la convergencia multiparadigma.

Tabla 1

Matriz Descriptiva de Paradigmas

Paradigma	Conceptos Clave	Gestión del Estado	Gestión del Estado	Aplicaciones
Imperativo/Estructural	Secuencia de instrucciones, variables, procedimientos	Estado mutable controlado explícitamente	Simplicidad y control directo	Sistemas de bajo nivel y aplicaciones generales
Orientado a Objetos	Encapsulamiento, herencia, polimorfismo, composición	Estado encapsulado en objetos	Reutilización y mantenibilidad	Sistemas empresariales y aplicaciones complejas
Funcional	Inmutabilidad, funciones de primer orden,	Estado inmutable	Menor cantidad de errores y mayor predictibilidad	Ciencia de datos y procesamiento paralelo

	transparencia referencial			
Lógico/Declarativo	Reglas, hechos, unificación e inferencia	Gestionado por el motor lógico	Facilita la representación del conocimiento	Inteligencia artificial y sistemas expertos
Concurrente/Actores	Actores, paso de mensajes, aislamiento de estado	Estado independiente por actor	Escalabilidad y reducción de condiciones de carrera	Sistemas distribuidos y aplicaciones de alta conurrencia

Análisis de los Paradigmas

El paradigma imperativo se basa en la modificación explícita del estado del sistema mediante una secuencia de instrucciones. Su principal fortaleza es el control directo sobre la ejecución del programa, aunque el uso excesivo de estados mutables puede dificultar el mantenimiento.

El paradigma orientado a objetos organiza el software mediante objetos que integran datos y comportamiento. Esto favorece la reutilización del código, la modularidad y la construcción de sistemas complejos.

El paradigma funcional promueve el uso de funciones puras e inmutabilidad de datos, reduciendo los efectos secundarios y facilitando el desarrollo de aplicaciones más seguras y predecibles.

El paradigma lógico se centra en la definición de reglas y relaciones, permitiendo que el sistema determine automáticamente cómo alcanzar una solución sin que el programador controle directamente el flujo de ejecución.

El paradigma concurrente o de actores propone la ejecución simultánea de múltiples entidades independientes que se comunican mediante mensajes, facilitando el desarrollo de sistemas escalables y distribuidos.

Convergencia Multiparadigma

Actualmente, los lenguajes modernos combinan características de varios paradigmas para ofrecer mayor flexibilidad. Python integra programación imperativa, orientada a objetos y funcional; JavaScript combina elementos imperativos, funcionales y orientados a objetos; mientras que Rust incorpora conceptos imperativos, funcionales y concurrentes. Esta convergencia multiparadigma permite seleccionar el enfoque más adecuado para cada problema, mejorando la productividad y adaptabilidad en el desarrollo de software moderno.

El paradigma lógico se centra en la definición de reglas y relaciones, permitiendo que el sistema determine automáticamente cómo alcanzar una solución sin que el programador controle directamente el flujo de ejecución.

El paradigma concurrente o de actores propone la ejecución simultánea de múltiples entidades independientes que se comunican mediante mensajes, facilitando el desarrollo de sistemas escalables y distribuidos.

Conclusión

Los paradigmas de programación proporcionan diferentes formas de abordar la construcción de software. Aunque cada uno posee características particulares, la tendencia actual apunta a la integración de múltiples paradigmas dentro de un mismo lenguaje. Esta convergencia permite aprovechar las fortalezas de cada enfoque y responder de manera más eficiente a los desafíos del desarrollo de software contemporáneo.

Actividad II. Estudio Morfológico, Sintáctico y Benchmarking

Análisis Morfológico (Léxico)

Zig emplea palabras reservadas como `if`, `while`, `fn`, `const`, `var` y `comptime`. Los identificadores pueden contener letras, números y guiones bajos, siempre que no comiencen con un número. Soporta literales enteros, flotantes, cadenas de caracteres y valores booleanos. La delimitación de bloques se realiza mediante llaves (`{ }`), lo que permite que los espacios y saltos de línea no tengan relevancia sintáctica.

Python utiliza palabras reservadas como `if`, `while`, `def`, `class` e `import`. Los identificadores siguen reglas similares a las de otros lenguajes modernos, permitiendo letras, números y guiones bajos. Admite literales numéricos, cadenas de texto y valores booleanos. Su característica más

distintiva es el uso de la indentación significativa para delimitar bloques de código, eliminando la necesidad de utilizar llaves o palabras de cierre.

Rust incorpora palabras reservadas como `if`, `while`, `fn`, `let`, `mut` y `match`. Los identificadores pueden construirse mediante letras, números y guiones bajos. Soporta literales enteros, flotantes, cadenas y booleanos. Al igual que Zig, utiliza llaves para delimitar bloques de código, permitiendo una separación clara entre las estructuras sintácticas del programa.

JavaScript emplea palabras reservadas como `if`, `while`, `function`, `let` y `const`. Sus identificadores siguen reglas similares a las de los demás lenguajes analizados. Admite literales numéricos, cadenas de caracteres, booleanos y otros tipos dinámicos propios del lenguaje.

La delimitación de bloques se realiza mediante llaves, facilitando la organización de estructuras de control y funciones.

Comparación General

Los cuatro lenguajes comparten una estructura léxica basada en identificadores, palabras reservadas y literales. Sin embargo, Python se diferencia significativamente al utilizar la indentación como mecanismo obligatorio para definir bloques de código, mientras que Zig, Rust y JavaScript emplean delimitadores explícitos mediante llaves. Estas diferencias influyen directamente en la forma en que los analizadores léxicos procesan y clasifican los componentes de cada programa.

Análisis Sintáctico

El análisis sintáctico tiene como finalidad verificar que la secuencia de tokens generada durante el análisis léxico cumpla las reglas gramaticales definidas por el lenguaje. Esta fase permite construir estructuras lógicas que representan la organización jerárquica del programa, tales como condicionales, bucles y definiciones de funciones.

Desde el punto de vista sintáctico, los cuatro lenguajes analizados permiten representar las mismas estructuras fundamentales de programación: condicionales, bucles y funciones. Sin embargo, cada uno adopta decisiones de diseño diferentes para expresar estas construcciones.

Python destaca por utilizar la indentación como elemento obligatorio para delimitar bloques de código. Esta característica favorece la legibilidad y obliga a mantener una estructura visual

consistente. En contraste, Zig, Rust y JavaScript utilizan llaves ({}) para definir explícitamente el inicio y fin de los bloques, permitiendo una mayor flexibilidad en el formato del código.

En la definición de funciones también existen diferencias sintácticas. Zig y Rust emplean la palabra clave `fn`, Python utiliza `def` y JavaScript la palabra reservada `function`. A pesar de estas variaciones, todas cumplen la misma finalidad: encapsular instrucciones reutilizables dentro de un subprograma.

Las estructuras condicionales y repetitivas presentan una sintaxis muy similar entre Zig, Rust y JavaScript, basada en expresiones lógicas seguidas por bloques delimitados mediante llaves. Python simplifica esta representación eliminando las llaves y sustituyéndolas por la indentación.

En términos generales, Zig y Rust muestran una sintaxis orientada al desarrollo de sistemas y al control explícito de la ejecución, JavaScript mantiene una sintaxis heredada de la familia C adaptada a entornos web y eventos, mientras que Python prioriza la simplicidad y legibilidad. A pesar de estas diferencias formales, todos los lenguajes permiten construir estructuras sintácticas equivalentes para resolver los mismos problemas computacionales.

Benchmarking de Rendimiento

Para evaluar el desempeño de los lenguajes analizados se implementó el algoritmo correspondiente a la Conjetura de Collatz. La prueba consistió en calcular el número de iteraciones necesarias para alcanzar el valor 1 para todos los enteros comprendidos entre 1 y 100.000.

Las ejecuciones fueron realizadas bajo condiciones controladas utilizando el mismo hardware y sistema operativo para cada lenguaje, con el propósito de minimizar factores externos que pudieran afectar los resultados.

El entorno de pruebas estuvo conformado por un equipo Lenovo con procesador AMD Ryzen de 2.1 GHz, 16 GB de memoria RAM y sistema operativo Windows 11 Pro de 64 bits. El tiempo de ejecución fue medido utilizando herramientas de temporización nativas de cada lenguaje.

Los resultados obtenidos permiten comparar el rendimiento relativo de las distintas implementaciones y analizar el impacto que tienen las decisiones de diseño de cada lenguaje sobre la eficiencia computacional.

Tabla 2*Matriz Descriptiva de Paradigmas*

Lenguaje	Paradigma Dominante	Ejecución y Compilación	Tiempo de ejecución (ms)	Pasos calculados
Python	Multiparadigma (Orientado a Objetos e Imperativo)	Interpretado mediante CPython y máquina virtual	1397,15	10.753.840
JavaScript (Node.js v24.15.0)	Multiparadigma (Prototípico y Funcional)	Compilación JIT (Just-In-Time) mediante V8 Engine	24,78	10.753.840

Nota. Las pruebas fueron realizadas sobre un equipo Lenovo con procesador AMD Ryzen de 2,1 GHz, 16 GB de memoria RAM y sistema operativo Windows 11 Pro de 64 bits. El algoritmo ejecutó la Conjetura de Collatz para todos los enteros comprendidos entre 1 y 100.000.

Los resultados evidencian diferencias significativas en el rendimiento de las implementaciones analizadas. Aunque ambos lenguajes produjeron exactamente la misma cantidad de iteraciones, la ejecución mediante Node.js registró un tiempo considerablemente menor que Python. Este comportamiento puede atribuirse a las optimizaciones realizadas por el motor V8, el cual incorpora técnicas de compilación Just-In-Time (JIT) que permiten transformar código frecuentemente ejecutado en instrucciones altamente optimizadas durante la ejecución. Por su parte, Python privilegia la simplicidad y productividad del desarrollo sobre el rendimiento puro, lo que explica la diferencia observada en esta prueba computacional.

Actividad III. Diseño del Lenguaje L para el Sistema ECO-GRID

Descripción General

El Lenguaje L es un Lenguaje de Dominio Específico (DSL) diseñado para la supervisión y control de microredes eléctricas inteligentes. Su objetivo es proporcionar a los operadores una sintaxis simple y legible para monitorear sensores, gestionar almacenamiento energético y controlar dispositivos físicos del sistema ECO-GRID.

Alfabeto y Reglas Léxicas

El analizador léxico del Lenguaje L reconoce los siguientes elementos:

Identificadores

- `bateria_1`
- `bateria_principal`
- `sector_industrial`
- `area_medica`
- `servidores_criticos`

Literales numéricos

- Números enteros: 20, 55, 100
- Números decimales: 90.5, 12.75

Literales booleanos

- `verdadero`
- `falso`

Operadores Relacionales

- `>`
- `<`
- `>=`
- `<=`
- `==`

Delimitadores

- `(`
- `)`
- `,`

Palabras Reservadas

- `init_grid`
- `leer_temperatura()`
- `estado_carga()`

- leer_generacion()
- leer_demanda()
- activar_refrigeracion()
- conmutar_linea()
- inyectar_red_publica()
- emitir_alerta()
- si
- entonces
- fin_si
- mientras
- ejecutar
- fin_mientras

Gramática Sintáctica Abstracta

Toda instrucción válida debe comenzar con una palabra reservada seguida de los parámetros requeridos entre paréntesis.

Las estructuras condicionales siguen la forma:

si condición entonces

acciones

fin_si

Las estructuras repetitivas siguen la forma:

mientras condición ejecutar

acciones

fin_mientras

Un programa puede contener múltiples instrucciones ejecutadas de manera secuencial dentro de un bloque principal iniciado mediante la instrucción `init_grid`.

Figura 1

Programa A: Prevención de Fuga Térmica y Gestión de Alivio de Carga

```
1  init_grid
2  mientras verdadero ejecutar
3      si leer_temperatura(bateria_1) > 55 entonces
4          activar_refrigeracion(bateria_1)
5          conmutar_linea(carga_solar, desconectada)
6          conmutar_linea(sector_industrial, red_publica)
7          emitir_alerta()
8      fin_si
9  fin_mientras
```

Figura 2

Programa B: Balance de Carga y Optimización Energética Autónoma

```
1  init_grid
2  si estado_carga(bateria_1) > 90 entonces
3      si leer_generacion() > leer_demanda() entonces
4          inyectar_red_publica()
5      fin_si
6  fin_si
7  si estado_carga(bateria_1) < 20 entonces
8      conmutar_linea(sector_no_esencial, desconectada)
9      conmutar_linea(area_medica, conectada)
10     conmutar_linea(servidores_criticos, conectada)
11 fin_si
```

Conclusiones

El estudio realizado permitió comprender la evolución y diversidad de los paradigmas de programación, identificando las características que distinguen a los enfoques imperativo, orientado a objetos, funcional, lógico y concurrente. Asimismo, se evidenció que los lenguajes modernos tienden a integrar múltiples paradigmas con el fin de ofrecer mayor flexibilidad y capacidad de adaptación a distintos problemas.

El análisis léxico y sintáctico realizado sobre diversos lenguajes permitió observar cómo cada uno implementa estructuras gramaticales particulares para expresar instrucciones y controlar el flujo

de ejecución. Estas diferencias demuestran la importancia de los componentes formales de un lenguaje en la claridad, mantenibilidad y eficiencia del desarrollo de software.

Los resultados obtenidos mediante las pruebas de benchmarking evidenciaron diferencias significativas de rendimiento entre los lenguajes analizados, mostrando la influencia que tienen los mecanismos de ejecución, compilación e interpretación sobre el desempeño de una misma solución algorítmica.

Finalmente, el diseño del Lenguaje L para el sistema ECO-GRID permitió aplicar los conceptos de análisis léxico, sintáctico y abstracción de lenguajes en un contexto práctico. La creación de un Lenguaje de Dominio Específico demostró cómo es posible desarrollar herramientas especializadas para facilitar la interacción entre operadores humanos y sistemas físico-computacionales complejos, reforzando la importancia del diseño de lenguajes como disciplina fundamental dentro de la ingeniería de software.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson.

Fowler, M. (2010). *Domain-specific languages*. Addison-Wesley Professional.

Python Software Foundation. (2025). Python documentation. <https://docs.python.org/3>

The Rust Project Developers. (2025). The Rust programming language. <https://doc.rust-lang.org/book/>

The Zig Software Foundation. (2025). Zig documentation. <https://ziglang.org/documentation/>